PROJECT REPORT OF CO-OP

Project at Industry (Module-2)

ON

Simulation of Scalable and Secure Backend Architectures for Resource-Constrained Web Applications: Review

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:
Shivanshu
Shwetank Dohroo
Shubham Goyal
Animesh Chaudhri

Supervised By:
Dr. Anshu Singla

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

DECLARATION

I hereby certify that the work which is being presented in the project report entitled "Simulation of Scalable and Secure Backend Architectures for Resource-Constrained Web Applications: Review" in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of <Name of the Mentor>. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Shubham Goyal - 2210992364
Animesh Chaudhri - 2210991266
Shwetank Dohroo - 2210992373
Shivanshu - 2210992330

Date: 08-05-2026

This is to certify that the above statement made by the candidates is correct to the best of my knowledge and belief.

Dr. Anshu Singla
Professor
Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our mentor and supervisor for their invaluable guidance, constant encouragement, and constructive feedback throughout the course of this project. Their expertise in backend architecture and distributed systems greatly shaped the direction and quality of this work.

We are deeply thankful to the Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Technology, for providing us with the necessary laboratory infrastructure, including the Dell OptiPlex 7080 machines used for experimentation, and for fostering an environment conducive to research and innovation.

We also extend our appreciation to our peers and colleagues who provided timely suggestions and moral support during the development and testing phases of this project.

Finally, we are grateful to our families for their unwavering support and encouragement throughout our academic journey.

ABSTRACT

Today's web applications face the dual challenge of supporting an ever-increasing number of concurrent users while maintaining robust security guarantees, all under resource constraints such as limited budgets and commodity hardware. Selecting the optimal backend architecture is especially critical for small development teams, startups, and research centers where both over-engineering and under-engineering carry significant consequences.

This project report presents an empirical evaluation of three widely used backend architectures — monolithic, microservices, and hybrid — with respect to scalability, performance, and security for resource-constrained web applications. The experiments were conducted on two Dell OptiPlex 7080 machines running Ubuntu 22.04 LTS, using Node.js, Express, PostgreSQL, MongoDB, Docker, and NGINX. Load testing was performed using the k6 tool at three virtual user (VU) levels: 50 VU (low), 200 VU (medium), and 500 VU (high).

Results indicate that the monolithic architecture achieves the lowest response time (42 ms) under low loads due to the absence of inter-service network overhead. However, its performance degrades severely at high concurrency, reaching an average response time of 1347 ms and a 4.71% error rate at 500 VUs. The microservices architecture supports the highest throughput (265 RPS at 500 VUs) but incurs significantly higher memory consumption and introduces security inconsistency risks due to decentralized authentication logic. The hybrid architecture, employing NGINX as a reverse proxy gateway for centralized JWT authentication and rate limiting, offers a well-balanced trade-off, achieving zero errors at 200 VUs and demonstrating superior security posture.

The findings suggest that the hybrid architecture is a pragmatic choice for resource-constrained teams seeking improved scalability without the operational complexity of a full microservices deployment.

Keywords: Backend Architecture, Scalability, Security, Microservices, Resource-Constrained Systems, Monolithic, Hybrid, Node.js, NGINX, Load Testing.

1. INTRODUCTION

The proliferation of web-based services has placed increasing strain on backend infrastructure. Developers must select a backend architecture that scales with fluctuating traffic while providing adequate security, often within strict budgetary and hardware constraints. This challenge is particularly acute for startups and academic research labs, where resources are finite and architectural decisions have long-term consequences.

Three dominant backend architectural paradigms exist in current practice: (1) the monolithic architecture, in which all backend components operate within a single process; (2) the microservices architecture, in which backend components are deployed as independent services communicating over a network; and (3) the hybrid architecture, which combines a monolithic backend with an API gateway that handles cross-cutting concerns such as authentication and rate limiting.

While prior research has compared monolithic and microservices architectures in terms of throughput and resource utilization, there remains a gap in studies conducted specifically under the constraints of shared commodity hardware and limited memory. This project addresses that gap by designing and executing controlled experiments to compare all three architectures across key performance and security metrics under low, medium, and high load conditions.

The objectives of this project are: (i) to empirically measure and compare response time, throughput, memory usage, CPU utilization, and HTTP error rates for monolithic, microservices, and hybrid architectures; (ii) to evaluate the security behavior of each architecture with respect to JWT validation, rate limiting, and handling of unauthorized access; and (iii) to derive practical recommendations for resource-constrained development teams.

2. METHODOLOGY

2.1 Application Design

The test application was structured into three functional modules shared across all three architecture implementations: (a) a data processing endpoint performing heavy write operations including insertions and aggregations; (b) a report endpoint executing query operations and returning aggregated data; and (c) an authentication module managing user registration, login, and JWT token issuance. The logic within each module was kept identical across all three architectural implementations to ensure a fair and unbiased comparison.

2.2 Hardware and Software Environment

Two Dell OptiPlex 7080 machines from the college laboratory were used. Both machines are equipped with an Intel Core i5-10400 processor (6 cores, 2.9 GHz base clock), 8 GB DDR4 RAM, and a 256 GB SSD, running Ubuntu 22.04 LTS. One machine hosted the application and database services, while the second was dedicated exclusively to k6 load generation. The machines were connected via a wired LAN. Tests were conducted during evening hours to minimize interference from shared network activity.

Software versions used: Node.js 18.17.1, Express 4.18.2, PostgreSQL 15.3, MongoDB 6.0.7, Docker 24.0.5, NGINX 1.24.0, PM2 5.3.0, and k6 v0.45.0. PM2 was used for process management uniformly across all configurations.

2.3 Architecture Configurations

Monolithic Architecture: All three functional modules operate as Express routers within a single Node.js process. PostgreSQL stores user and session data; MongoDB stores processed records and reports. PM2 was applied to all configurations uniformly after the application exhibited crashes under high loads during initial testing.

Microservices Architecture: Each of the three functional modules — authentication, data processing, and reporting — runs as an independent Node.js application in a separate Docker container. Services communicate via REST over a Docker user-defined bridge network with static IP addresses, which

resolved initial DNS resolution latency spikes of 5–15 ms observed with the default bridge network. Docker Compose memory limits of 512 MB per container were applied after the combined RSS of all containers approached 7 GB on the 8 GB machine during initial testing, causing the OS to swap.

Hybrid Architecture: The same Node.js monolithic application is deployed behind an NGINX reverse proxy. NGINX performs JWT authentication using the ngx_http_auth_jwt_module and enforces rate limiting via limit_req_zone (100 requests/second per IP; HTTP 429 returned for excess requests). The Node.js application layer does not perform any authentication, as all requests are pre-validated at the gateway. TLS termination is handled at the NGINX layer, which is noted as a limitation when directly comparing security metrics with the other two architectures.

2.4 Load Testing Protocol

Load testing was conducted using the k6 tool at three virtual user (VU) levels: 50 VUs (low load), 200 VUs (medium load), and 500 VUs (high load). Each level was tested three times and the results averaged. Each load level ran for 3 minutes at a constant VU count, with a 30-second ramp-up and ramp-down period. The k6 test script simulated realistic user workflows: authentication and JWT token acquisition, followed by five consecutive data processing requests and two report requests. Metrics captured included average response time, 95th percentile response time, requests per second (RPS), RSS memory usage, CPU usage, and HTTP error rate. CPU and memory usage were logged every 5 seconds using a custom monitoring script.

3. TOOLS AND TECHNOLOGIES

The following tools and technologies were employed in the design, development, and evaluation of the three backend architectures:

Node.js (v18.17.1): A JavaScript runtime built on Chrome's V8 engine, used as the primary backend language for all three architecture implementations. Its non-blocking, event-driven model makes it well-suited for handling concurrent web requests.

Express.js (v4.18.2): A minimal and flexible Node.js web application framework used for routing and middleware management within all three architectures.

PostgreSQL (v15.3): An open-source relational database management system used for storing user credentials and session data. Connection pooling behavior was intentionally left at default settings to observe untuned performance characteristics.

MongoDB (v6.0.7): A NoSQL document-oriented database used for storing processed data records and aggregated reports.

Docker (v24.0.5) and Docker Compose: Containerization technology used to isolate and deploy individual microservices. Docker user-defined bridge networks with static IP addresses were configured to eliminate per-call DNS resolution overhead.

NGINX (v1.24.0): A high-performance reverse proxy and web server used in the hybrid architecture to handle JWT authentication (ngx_http_auth_jwt_module) and rate limiting (limit_req_zone). Acts as the centralized API gateway layer.

PM2 (v5.3.0): A process manager for Node.js applications, applied uniformly across all architectures to handle process restarts and ensure consistent runtime behavior during load testing.

k6 (v0.45.0): A modern open-source load testing tool used to simulate concurrent virtual users. Enabled precise control over VU counts, ramp-up/ramp-down periods, and scripted user workflows.

4. IMPLEMENTATION

4.1 Monolithic Implementation

In the monolithic configuration, all three modules — authentication, data processing, and reporting — were implemented as Express routers mounted within a single Node.js application process. A single instance of PostgreSQL and MongoDB served all data persistence needs. The application was managed by PM2 to ensure automatic process recovery on crashes. This architecture required no inter-process communication, resulting in minimal internal latency under low load conditions.

A critical issue encountered during implementation was PostgreSQL connection pool exhaustion at medium load. With the default pool size of 10 connections, the concurrent authentication requests from 200 VUs exceeded available connections, resulting in queued requests and a 0.33% error rate. This configuration was preserved intentionally to reflect untuned real-world behavior.

4.2 Microservices Implementation

Each of the three modules was containerized as an independent Docker image and orchestrated via Docker Compose. The services communicated over a user-defined Docker bridge network. During initial testing, DNS resolution per inter-container call introduced latency spikes of 5–15 ms; this was resolved by assigning static IP addresses within the Docker network. Memory limits of 512 MB per container were enforced via Docker Compose to prevent memory exhaustion on the 8 GB test machine.

A notable security vulnerability was discovered during testing: the data processing microservice validated the JWT signature but failed to check the exp (expiration) claim, accepting tokens that had expired approximately three hours prior. This defect illustrates a key risk of decentralized security implementation in microservices architectures.

4.3 Hybrid Implementation

The hybrid configuration deployed the same monolithic Node.js application behind an NGINX reverse proxy acting as an API gateway. NGINX was configured with the ngx_http_auth_jwt_module to perform JWT validation on all incoming requests before forwarding them to the application layer. Rate limiting was enforced at the gateway level using limit_req_zone, capping requests to 100 per second per IP address and returning HTTP 429 for excess requests. The Node.js application layer was relieved of all authentication responsibilities, simplifying application-level code and eliminating the risk of inconsistent token validation across service boundaries.

NGINX and the Node.js process were co-hosted on the same machine, introducing potential CPU contention under high load. This is acknowledged as a limitation of the experimental setup and may have contributed to the relatively slower response times observed at 500 VUs compared to microservices.

5. MAJOR FINDINGS

5.1 Response Time and Throughput

Table 1 presents the complete set of performance measurements for all three architectures across all three load levels, averaged over three runs.

Table 1: Response Time and Throughput at Three Load Levels (Averages over three runs)

Architecture	Load (VUs)	Avg RT (ms)	p95 RT (ms)	RPS	Error (%)
Monolith	50	42	73	117	0.00
Microservices	50	63	101	80	0.00
Hybrid	50	55	86	91	0.00
Monolith	200	191	423	103	0.33
Microservices	200	129	207	154	0.00
Hybrid	200	146	241	137	0.00
Monolith	500	1347	3210	69	4.71
Microservices	500	188	318	265	0.19
Hybrid	500	258	503	194	0.51

At 50 VUs (low load), all three architectures operated error-free. The monolith recorded the lowest average response time of 42 ms, owing to the absence of inter-service communication overhead. Microservices averaged 63 ms due to REST communication between containers, and the hybrid architecture averaged 55 ms.

At 200 VUs (medium load), the monolithic architecture began exhibiting degradation. Average response time rose to 191 ms with a 0.33% error rate, caused by PostgreSQL connection pool exhaustion. Microservices recorded a lower average of 129 ms with zero errors, and the hybrid architecture averaged 146 ms with zero errors.

At 500 VUs (high load), the monolith experienced severe degradation: average response time reached 1347 ms, the 95th percentile exceeded 3.2 seconds, and the error rate climbed to 4.71%. The Node.js event loop was observed operating at 88–92% of a single CPU core, consistent with saturation of the single-threaded model under heavy concurrent queuing. Microservices sustained an average of 188 ms with an RPS of 265 and only a 0.19% error rate. The hybrid architecture averaged 258 ms at 194 RPS with a 0.51% error rate.

5.2 Memory and CPU Utilization

At 500 VUs, the three microservices containers collectively consumed approximately 840–870 MB of RSS memory, compared to approximately 410 MB for the monolith and 590 MB for the hybrid configuration (inclusive of the NGINX process). Microservices therefore consumed roughly twice the memory of the monolith under equivalent load conditions, representing a significant resource trade-off for memory-constrained environments.

5.3 Security Behavior

Three security dimensions were evaluated: rejection of unauthorized JWTs, enforcement of rate limiting, and handling of repeated failed authentication attempts.

In the monolith, JWT validation was handled by Express middleware before each protected endpoint. All 200 invalid or unauthorized tokens tested correctly received HTTP 401 responses. The primary security risk identified was the potential for security misconfiguration within the application code.

In the microservices architecture, a security defect was discovered in the data processing service: it validated the JWT signature but neglected to check the exp claim, thereby accepting expired tokens. This represents a significant vulnerability and highlights the danger of independently implementing security logic across services.

In the hybrid architecture, all authentication and rate limiting logic was handled exclusively by NGINX at the gateway layer. Invalid, expired, and missing tokens uniformly returned HTTP 401;

rate-limited requests returned HTTP 429. Centralizing security logic at the gateway eliminated the risk of inconsistencies across service boundaries and provided the most robust security posture of the three architectures evaluated.

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project has empirically evaluated the performance, scalability, and security behavior of monolithic, microservices, and hybrid backend architectures under controlled load conditions on commodity hardware. The following key conclusions are drawn from the experimental results:

The monolithic architecture achieves the best performance under low load conditions, with an average response time of 42 ms at 50 VUs and zero errors. However, it degrades significantly under high concurrency, exhibiting a 4.71% error rate and an average response time exceeding 1.3 seconds at 500 VUs. Default connection pool limits and single-threaded execution are the primary bottlenecks.

The microservices architecture provides the highest throughput at high load (265 RPS at 500 VUs) but demands approximately twice the memory of the monolith and introduces security risks when authentication logic is implemented independently per service. The discovered JWT expiration vulnerability illustrates a real-world security pitfall of the decentralized model.

The hybrid architecture offers a compelling balance: zero errors at medium load (200 VUs), competitive throughput at high load, and the strongest security posture due to centralized gateway-level authentication and rate limiting via NGINX. The JWT expiration vulnerability present in the microservices implementation would have been prevented entirely in the hybrid configuration. For resource-constrained teams seeking a path toward improved scalability without the full operational overhead of microservices, the hybrid architecture represents a practical and operationally simpler alternative.

6.2 Future Scope

Future work should explore the following directions to extend and validate the findings of this study: (i) replication of experiments on dedicated hardware per component to eliminate CPU contention between the NGINX gateway and Node.js application in the hybrid configuration; (ii) application of production-grade database configurations including tuned connection pool sizes, indexing, and

caching to evaluate performance without the constraints of default settings; (iii) implementation of database-per-service isolation in the microservices configuration, as prescribed by microservices best practices; (iv) extension of the security evaluation to include TLS benchmarking across all three architectures under comparable conditions; and (v) evaluation of serverless and container-as-a-service platforms as alternative deployment models for resource-constrained teams.

REFERENCES

[1] G. Blinowski, A. Ojdowska, and A. Przybyłek, "Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation," IEEE Access, vol. 10, pp. 20357–20374, 2022, doi: 10.1109/ACCESS.2022.3152803.

[2] O. Al-Debagy and P. Martinek, "A Comparative Review of Microservices and Monolithic Architectures," IEEE 18th International Symposium on Computational Intelligence and Informatics (CINTI), Budapest, Hungary, 2018, pp. 000149–000154, doi: 10.1109/CINTI.2018.8928192.

[3] I. Čilić, P. Krivić, I. Podnar Žarko, and M. Kušek, "Performance Evaluation of Container Orchestration Tools in Edge Computing Environments," Sensors, vol. 23, p. 4008, 2023, doi: 10.3390/s23084008.

[4] R. Vaño, I. Lacalle, P. Sowiński, R. S-Julián, and C. E. Palau, "Cloud-Native Workload Orchestration at the Edge: A Deployment Review and Future Directions," Sensors, vol. 23, p. 2215, 2023, doi: 10.3390/s23042215.

[5] F. Tapia et al., "From Monolithic Systems to Microservices: A Comparative Study of Performance," Applied Sciences, vol. 10, p. 5797, 2020, doi: 10.3390/app10175797.

[6] A. Bucko, K. Vishi, B. Krasniqi, and B. Rexha, "Enhancing JWT Authentication and Authorization in Web Applications Based on User Behavior History," Computers, vol. 12, p. 78, 2023, doi: 10.3390/computers12040078.

[7] R. Su, X. Li, and D. Taibi, "From Microservice to Monolith: A Multivocal Literature Review," Electronics, vol. 13, p. 1452, 2024, doi: 10.3390/electronics13081452.

[8] S. du Plessis and N. Correia, "A Comparative Study of Software Architectures in Constrained Device IoT Deployments," IEEE International Conference on Internet of Things and Intelligence Systems (IoTaIS), 2021.

[9] M. Țălu, "A Comparative Study of WebAssembly Runtimes: Performance Metrics, Integration Challenges, Application Domains, and Security Features," Archives of Advanced Engineering Science, 2025.

[10] M. Abdelsattar et al., "Comparative Analysis of Deep Learning Architectures in Solar Power Prediction," Scientific Reports, vol. 15, p. 31729, 2025.

[11] E. Mabotha, N. E. Mabunda, and A. Ali, "A Dockerized Approach to Dynamic Endpoint Management for RESTful Application Programming Interfaces in Internet of Things Ecosystems," Sensors, vol. 25, p. 2993, 2025.

[12] N. A. Nguyen, "Comparison of WebAssembly and Container Technologies," 2025.

[13] M. M. Raikar et al., "Leveraging Docker Containers for Deployment of Web Applications in Microservices Architecture," First International Conference for Women in Computing (InCoWoCo), IEEE, 2024.

[14] K. Aruna and P. Gurunathan, "Enhancing Edge Environment Scalability: Leveraging Kubernetes for Container Orchestration and Optimization," Concurrency and Computation: Practice and Experience, vol. 36, p. e8303, 2024.

[15] J. Sander et al., "On Accelerating Edge AI: Optimizing Resource-Constrained Environments," arXiv preprint arXiv:2501.15014, 2025.